

A 3.2.4 Constructing tables

Introduction

In relational databases, data is organised into tables. How these tables are structured and how they relate to each other is fundamental to the database's effectiveness. This section explores the principles of table construction and the crucial role of keys in defining relationships.

Constructing Tables for Relational Databases

A table in a relational database is composed of rows (also called records) and columns (also called fields). Each column stores a specific type of data (as we discussed in A.3.2.3), and each row represents a single instance of the entity the table describes.

Key Considerations for Table Construction:

Entity Focus: Each table should represent a single, well-defined entity (e.g., "Customers," "Orders," "Products"). For example:

Customers
CustomerID
FirstName
LastName
Email
PhoneNumber

Atomic Columns: Columns should store atomic (indivisible) data. For example:

- **Instead of:** `Name (VARCHAR)`
- **Use:** `FirstName (VARCHAR), LastName (VARCHAR)`

Data Type Integrity: Each column must have a defined data type, and all data in that column must adhere to that type. For example:

Column	Data Type
Quantity	INTEGER
Price	DECIMAL
OrderDate	DATE

No Redundancy: Avoid storing the same data in multiple tables (this is addressed by normalization, a more advanced topic).

The Relationship Between Tables

Relational databases derive their power from the relationships between tables. These relationships are established and managed using keys:

Primary Key:

- A primary key is a column (or a set of columns) that uniquely identifies each row in a table.
- It must contain unique values.
- It cannot contain NULL (empty) values.
- Example: In a "Customers" table, "CustomerID" is often the primary key.

Customers
CustomerID (Primary Key)
FirstName
LastName
Email
PhoneNumber

Foreign Key:

- A foreign key is a column (or set of columns) in one table that refers to the primary key in another table.
- It establishes a link between the two tables.
- Example: In an "Orders" table, "CustomerID" would be a foreign key, linking each order to the customer who placed it in the "Customers" table.

Orders
OrderID (Primary Key)
CustomerID (Foreign Key)
OrderDate
TotalAmount

Composite Key:

- A composite key is a primary key that consists of two or more columns.
- It's used when a single column cannot uniquely identify a row.
- Example: In a table tracking student enrollment in courses, the composite key might be ("StudentID," "CourseID") since a student can enroll in multiple courses, and a course has multiple students.

Enrollments	
StudentID	Composite Key
CourseID	
EnrollmentDate	
Grade	

Concatenated Key

In some contexts, "concatenated key" might refer to the *value* formed by combining multiple columns (not strictly a key *type* like the others).

More accurately, if those concatenated values are used as a key, it is a composite key.

Example: If you combine "FirstName" and "LastName" to search for a record, you're using a concatenated *value*, but if those two columns *together* are the table's unique identifier, it's a composite key.

The Importance of Well-Defined Tables in Ensuring Data Integrity

Well-defined tables are crucial for maintaining data integrity, which refers to the accuracy and consistency of data stored in the database.

- **Benefits of Well-Defined Tables:**

- **Reduced Redundancy:** Proper table design minimizes data duplication, saving storage space and preventing inconsistencies.
- **Data Accuracy:** Constraints (like primary key uniqueness and data types) enforce rules about the data that can be entered, reducing errors.
- **Consistency:** Relationships defined by keys ensure that data across tables is consistent. For example, a foreign key ensures that you can't have an order referring to a non-existent customer.
- **Efficient Retrieval:** Well-structured tables make it easier and faster to retrieve the needed information.
- **Ease of Maintenance:** Changes to the database schema are easier to manage when tables are well-defined.

Key Questions

Here are some questions to check your understanding:

What is the purpose of a primary key in a relational database table?

The purpose of a primary key is to uniquely identify each row in a table. It ensures that no two rows are exactly the same, allowing for efficient retrieval and manipulation of data.

Explain how a foreign key is used to create a relationship between two tables.

A foreign key in one table "points" to the primary key in another table. This creates a link, allowing you to relate data from the two tables. For example, an "Orders" table might have a foreign key referencing the "Customers" table to show which customer placed each order.

When would you use a composite key instead of a single-column primary key?

You would use a composite key when no single column can uniquely identify each row in a table. This often happens in tables that represent relationships themselves. For instance, in a table tracking which students are enrolled in which courses, the combination of "StudentID" and "CourseID" is needed to uniquely identify each enrollment record.

Describe two ways that well-defined tables contribute to data integrity.

Reducing redundancy: This minimizes the chance of inconsistencies; if data is stored only once, updates are less likely to create conflicting information.

Enforcing constraints: Primary keys enforce uniqueness, and data types enforce what kind of data can go in a column, both preventing errors.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**